

Guided Project: Building a Spam Filter with Naive Bayes

Filtro de Spam de SMS através do algoritmo Naive Bayes

Durante esse módulos aprendemos diversos conceitos, fórmulas e seus usos a respeito da probabilidade. Aqui vamos consolidar todo esse conhecimento de fato criando um filtro de spam de SMS.

Para classificar como spam ou não:

- humanos precisam classificar algumas mensagens como spam ou não
- o sistema vai calcular as probabilidade a partir dessas mensagens já classificadas
- e com o resultado do cálculo será capaz de classificar novas mensagens

Para isso, o dataset a ser utilizado é baseado num dataset com +5000 mensagens de SMS que pode ser encontrado **aqui**.

No entanto vamos seguir com uma versão reduzida para diminuir o tempo de execução.

```
base <- read_csv("https://dq-content.s3.amazonaws.com/475/spam.csv")
```

```
## Rows: 1000 Columns: 2
## -- Column specification -----
## Delimiter: ","
## chr (2): label, sms
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# base <- read_tsv("C:/Users/marce/Downloads/sms+spam+collection/SMSSpamCollection")
glimpse(base)
```

```
## Rows: 1,000
## Columns: 2
## $ label <chr> "ham", "ham", "spam", "ham", "ham", "ham", "ham", "ham", "ham", "ham", ~
## $ sms <chr> "Ü mean it's confirmed... I tot they juz say oni... Ok then...",~
```

É uma base que possui apenas duas colunas, uma sendo “label” que é a classificação prévia dessas mensagens se é “spam” ou não (ham), e a coluna “sms” que é a própria mensagem em si. A base possui apenas 15% de suas mensagens classificadas como spam.

```
base %>%
  group_by(label) %>%
  summarise(row=n()) %>%
  mutate(perc = row/sum(row))
```

```
## # A tibble: 2 x 3
##   label   row perc
##   <chr> <int> <dbl>
## 1 ham     850  0.85
## 2 spam    150  0.15
```

Nosso projeto será um problema de machine learning, de classificação, e nosso objetivo é maximizar a habilidade preditiva do nosso algoritmo. Ou seja queremos que com base na experiência passada o algoritmo seja capaz de acertar o máximo possível a classificação de novas mensagens.

Para ensinar o algoritmo a classificar as mensagens, basicamente a classificação se dá através de duas equações que vimos anteriormente:

$$P(\text{Spam}|p_1, p_2, \dots, p_n) \propto P(\text{Spam}) * \prod_{i=1}^n P(p_i|\text{Spam})$$

$$P(\text{Ham}|p_1, p_2, \dots, p_n) \propto P(\text{Ham}) * \prod_{i=1}^n P(p_i|\text{Ham})$$

Trocamos o Spam^C para Ham que é a classificação oposta no nosso caso.

Para cada um $P(p_i|\text{Spam})$ ou $P(p_i|\text{Ham})$ temos uma fórmula, vamos relembrar o uso do *additive smoothing*.

$$P(p_i|\text{Spam}) = \frac{Np_i|\text{Spam} + \alpha}{N\text{Spam} + \alpha * NVocabulary}$$

$$P(p_i|\text{Ham}) = \frac{Np_i|\text{Ham} + \alpha}{N\text{Ham} + \alpha * NVocabulary}$$

Vamos relembrar o significado de cada notação:

- $Np_i|Spam$ - é o número de vezes que aparece a palavra p_i dado que é spam
- $Np_i|Ham$ - é o número de vezes que aparece a palavra p_i dado que é não spam
- $NSpam$ - total de palavras em mensagens que são spam
- $NHam$ - total de palavras em mensagens que não são spam
- $NVocabulary$ - total de palavras únicas no total independente da classificação
- α - é o parâmetro de *smoothing*

Divisão da base

Para começar precisamos dividir a base em 3 partes

- training set: é a parte da base que vamos usar para treinar o algoritmo e representará 80% da base total
- cross validation set: nós vamos usar para avaliar como as diferentes escolhas de α afetam a acurácia, será 10%
- test set: os 10% restantes da base que vamos verificar o quanto o algoritmo é bom ao classificar novas mensagens

```
indices <- sample(seq(1,nrow(base)), size=(0.8*nrow(base)))
```

```
length(indices)
```

```
## [1] 800
```

```
indices %>% head(20)
```

```
## [1] 437 233 853 189 882 378 652 249 632 425 888 540 552 55 224 316 306 344 415  
## [20] 366
```

```
training_set <- base[indices,]  
remaining <- base[-indices,]
```

```
indices_remain <- sample(seq(1,nrow(remaining)), size=(0.5*nrow(remaining)))
```

```
length(indices_remain)
```

```
## [1] 100
```

```
indices_remain %>% head(20)
```

```
## [1] 143 175 127 125 131 182 56 72 36 13 17 1 164 134 159 64 114 79 91  
## [20] 99
```

```
cross_validation_set <- remaining[indices_remain,]  
test_set             <- remaining[-indices_remain,]
```

O tamanho dos datasets ficaram de acordo com o planejado, 80/10/10

```
training_set %>% nrow()
```

```
## [1] 800
```

```
cross_validation_set %>% nrow()
```

```
## [1] 100
```

```
test_set %>% nrow()
```

```
## [1] 100
```

E a proporção de spam/ham foi num geral preservada.

```
mean(training_set$label == "spam")
```

```
## [1] 0.15875
```

```
mean(cross_validation_set$label == "spam")
```

```
## [1] 0.12
```

```
mean(test_set$label == "spam")
```

```
## [1] 0.11
```

Limpeza dos dados

Antes de começar a contar as palavras vamos precisar limpar a base de treino, pois as mensagens por serem livres tem pontuação, palavras com letras maiúsculas e minúsculas, e essas coisinhas podem atrapalhar nosso cálculo.

```
training_set <-
  training_set %>%
  # head(100) %>%
  mutate(sms_bkp = sms) %>%
  mutate(sms = str_to_lower(sms)) %>% #transformar em minúsculo
  mutate(sms = str_replace_all(sms, c("[:digit:]" = "", #remover pontos
    "[:punct:]" = "", #remover dígitos
    "\t" = "", #remover tab
    "\n" = "", #quebra de linha
    "\u0094" = "", #caracteres unicode
    "\u0092" = "",
    "\u0096" = "",
    "\u0091" = "",
    "\\+" = "", #símbolos
    "\\&" = "",
    "\\|" = "",
    "\\>" = ""))) %>%
  mutate(sms = str_squish(sms)) #remover espaços excedentes

training_set %>% head()
```

```
## # A tibble: 6 x 3
##   label sms sms_bkp
##   <chr> <chr> <chr>
## 1 ham lol oh no babe i wont be sliding into your place after midnight~ Lol ..~
## 2 ham staffsciencenusedusg~phyhcmkteachingpc staff.~
## 3 spam email alertfrom jeri stewartsize kbsubject lowcost prescriptiton~ Email ~
## 4 ham feel yourself that you are always happy slowly it becomes your ~ Feel Y~
## 5 ham no we put party days a week and study lightly i think we need t~ No we ~
## 6 ham i am on the way to ur home I am o~
```

Criando Vocabulário

Agora que nossa base de treino passou pela limpeza acima, podemos contar a quantidade de palavras únicas para criar nosso vocabulário.

```

vocabulario <- c()
mensagens <- training_set %>% pull(sms)

for (m in mensagens){
  palavras <- str_split(m, " ")[[1]] #extraíndo da lista
  vocabulario <- c(vocabulario, palavras)
}

#deduplicando
vocabulario <- unique(vocabulario)

```

Calculando constantes

Antes de seguir para o cálculo final, vamos calcular as constantes que são utilizadas na fórmula como N_{Spam} , N_{Ham} e $N_{Vocabulary}$. Começando por determinar o valor do $\alpha = 1$, por hora não sabemos se esse valor é o melhor para maximizar o resultado, mais ao final vamos revisitar esse ponto.

Primeiro vamos separar a base de treino em 2, base somente com Spam, e base sem spam.

```

training_setSpam <- training_set %>% filter(label == "spam")
training_setHam <- training_set %>% filter(label == "ham")

```

Agora vamos calcular a quantidade de palavras de cada categoria, porém agora sem deduplicar.

```

vocabularioSpam <- c()
mensagensSpam <- training_setSpam %>% pull(sms)

for (m in mensagensSpam){
  palavras <- str_split(m, " ")[[1]] #extraíndo da lista
  vocabularioSpam <- c(vocabularioSpam, palavras)
}

```

```

vocabularioHam <- c()
mensagensHam <- training_setHam %>% pull(sms)

for (m in mensagensHam){
  palavras <- str_split(m, " ")[[1]] #extraíndo da lista
  vocabularioHam <- c(vocabularioHam, palavras)
}

```

```
NSpam      <- vocabularioSpam %>% length()
NHam       <- vocabularioHam  %>% length()
NVocabulary <- vocabulario     %>% length()
```

```
NSpam
```

```
## [1] 5296
```

```
NHam
```

```
## [1] 20043
```

```
NVocabulary
```

```
## [1] 5023
```

Calculando Probabilidades

Agora que calculamos as constantes, vamos conseguir calcular as duas grandes probabilidades $P(\textit{Spam})$ e $P(\textit{Ham})$. Com esse resultado vamos conseguir calcular a probabilidade condicional.

```
p_spam <- mean(training_set$label == "spam")
p_ham  <- mean(training_set$label == "ham")
```

```
p_spam
```

```
## [1] 0.15875
```

```
p_ham
```

```
## [1] 0.84125
```

Repare que ficou bem parecido com a proporção de spam/ham do início do exercício, o que é um bom indicativo que a amostragem deu certo e é equivalente a base original. Calcular a média é equivalente a contar quantas mensagens spam dividido pelo total. A condição `training_set$label == "spam"` retorna 0 ou 1 e a média vai somar todos os 1s e dividir pelo total chegando no mesmo resultado.

Agora vamos à probabilidade de cada palavra, lembrando que p_i representa cada palavra, e os passos para calcular a palavra na base de Spam ou Ham será idêntico. Uma vez que nossa base de treino não muda, a probabilidade da palavra “secret” dado que é Spam será sempre a mesma, constante, o mesmo com acontece para não Spam.

Para chegar em nossa resposta final, temos que antes calcular a probabilidade de cada palavra da mensagem dado que é spam e o mesmo dado que não é spam. Então se a mensagem é spam e é composta por “secret,”code”, “unlock”, temos que achar $P(\text{"secret"}|Spam)$, $P(\text{"code"}|Spam)$ e $P(\text{"unlock"}|Spam)$. Esses valores sendo calculados chamamos de **parâmetros**, e estamos usando uma amostra (a base de treino) para estimar o valor, visto que não sabemos seu valor “real”. O valor vai variar dependendo da seleção na base de treino, e torcemos para que a base seja representativa o suficiente para o valor ser o mais próximo do real.

O código vai para cada palavra presente no Vocabulário Spam/Ham ver se encontra em cada mensagem e quantas vezes por mensagem e consolidar num valor final.

```
# Criando uma base para a contagem de palavras presentes em spam
contagem_spam <-
  tibble(palavra = vocabularioSpam) %>%
  mutate(spam_count = map_int(palavra,
                                function(p) {
                                  map_int(training_set$spam$sms,
                                            function(spam) {
                                              (str_split(spam, " ")[[1]] == p) %>% sum
                                            }) %>% sum
                                })
  )
```

```
# Criando uma base para a contagem de palavras presentes em ham
contagem_ham <-
  tibble(palavra = vocabularioHam) %>%
  mutate(ham_count = map_int(palavra,
                              function(p) {
                                map_int(training_set$ham$sms,
                                          function(ham) {
                                            (str_split(ham, " ")[[1]] == p) %>% sum
                                          }) %>% sum
                              })
```

```
)
    })
```

```
contagem_palavras <- full_join(contagem_spam,
                                contagem_ham,
                                by="palavra") %>%
  mutate(spam_count = if_else(is.na(spam_count), 0, spam_count),
         ham_count = if_else(is.na(ham_count), 0, ham_count))
```

```
## Warning in full_join(contagem_spam, contagem_ham, by = "palavra"): Detected an unequ
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 9883 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship =
## "many-to-many"` to silence this warning.
```

O cálculo prévio de todas essas constantes e parâmetros torna o algoritmo Naive Bayes bem rápido, pois quando uma nova mensagem chegar, a maior parte dos cálculos estarão prontos.

Classificando uma nova mensagem

Agora com todos parâmetros e constantes calculadas retornamos a fórmula inicial para enfim calcular o resultado final:

$$P(p_i|Spam) = \frac{Np_i|Spam + \alpha}{NSpam + \alpha * NVocabulary}$$

$$P(p_i|Ham) = \frac{Np_i|Ham + \alpha}{NHam + \alpha * NVocabulary}$$

Lembrando que...

- $Np_i|Spam$ é a quantidade de vezes que a palavra p_i aparece em mensagens de spam
- $Np_i|Ham$ é a quantidade de vezes que a palavra p_i aparece em mensagens que não são spam

E esse resultado nos permite encontrar o resultado para $P(p_i|Spam)$ e $P(p_i|Ham)$ que são informações chave para as fórmulas:

$$P(Spam|p_1, p_2, \dots, p_n) \propto P(Spam) * \prod_{i=1}^n P(p_i|Spam)$$

$$P(Ham|p_1, p_2, \dots, p_n) \propto P(Ham) * \prod_{i=1}^n P(p_i|Ham)$$

```
classificar_mensagem <- function(msg, alpha = 1){

  mensagem_tratada <-
    str_to_lower(msg) %>%
    str_replace_all( c("[:digit:]" = "",
                      "[:punct:]" = "",
                      "\t" = "",
                      "\n" = "",
                      "\u0094" = "",
                      "\u0092" = "",
                      "\u0096" = "",
                      "\u0091" = "",
                      "\\+" = "",
                      "\\&" = "",
                      "\\|" = "",
                      "\\>" = "")) %>%

    str_squish()

  #extraíndo palavras da msg
  palavras <- str_split(mensagem_tratada, " ")[[1]]

  #conferindo palavras novas que não pertencem ao vocabulário ainda
  novas_palavras <- setdiff(vocabulario, palavras)

  prob_palavras_novas <- tibble(
    palavra = novas_palavras,
    spam_prob = 1,
    ham_prob = 1)

  #calculando a probabilidade das palavras dado que é spam/ham
  prob_palavras_existentes <-
    contagem_palavras %>%
    filter(palavra %in% palavras) %>%
    mutate(
      spam_prob = (spam_count + alpha) / (NSpam + alpha * NVocabulary),
```

```

    ham_prob = (ham_count + alpha) / (NHam + alpha * NVocabulary)
  ) %>%
  bind_rows(prob_palavras_novas) %>%
  pivot_longer(
    cols = c("spam_prob", "ham_prob"),
    names_to = "classificacao",
    values_to = "prob"
  ) %>%
  group_by(classificacao) %>%
  summarise(
    pi_prob = prod(prob)) #função de multiplicação igual ao sum

#calculando probabilidade condicional
p_spam_dado_mensagem <- p_spam * (prob_palavras_existentes %>%
  filter(classificacao == "spam_prob") %>%
  pull(pi_prob))

p_ham_dado_mensagem <- p_ham * (prob_palavras_existentes %>%
  filter(classificacao == "ham_prob") %>%
  pull(pi_prob))

#classificar com base no resultado
ifelse(p_spam_dado_mensagem >= p_ham_dado_mensagem, "spam", "ham")
}

#vamos utilizar a função criada na própria base de treino

training_set <- training_set %>%
  mutate(predicao = map_chr(sms, function(mensagem) {classificar_mensagem(mensagem)}))

```

Calculando a Acurácia

Agora que criamos uma nova coluna com a decisão do algoritmo temos que conferir o quanto ele está acertando. Uma forma é calculando a acurácia através da divisão entre a quantidade de classificações certas pela quantidade de classificações no total.

Com a função `table` criamos uma **matriz de confusão** que nos permite com facilidade ver quais mensagens de spam eram realmente spam, e quais mensagens ham eram ham, versus as demais combinações equivocadas.

```
matriz <- training_set %>% select(label, predicao) %>% table()
matriz
```

```
##      predicao
## label  ham spam
##   ham   13  660
##   spam    0  127
```

```
 #(acertos spam+acertos ham) / (total)
acuracia <- (matriz[1,1] + matriz[2,2]) / sum(matriz)
acuracia
```

```
## [1] 0.175
```

Hyperparameters Tuning e Cross-Validation

Até aqui nós escolhemos arbitrariamente o valor $\alpha = 1$ para o nosso additive smoothing, mas não sabemos se é esse valor que produz as melhores previsões. o α é um parâmetro que podemos alterar (*tune* do inglês, ou “aportuguesando” é comum dizer “tunar”) com o objetivo de aprimorar a previsão. Parâmetros como esse são chamados de **hiperparâmetros**.

Para saber qual o melhor valor usar precisamos avaliar como isso afeta a previsão. Mas não podemos usar a base de treino, pois ela já é utilizada para estimar outros parâmetros, então para ter uma estimativa mais precisa, precisamos aplicar numa base ainda não utilizada.

Para isso, podemos utilizar o cross validation set que separamos no começo do projeto. Não vamos utilizar o test set até que estejamos no final do processo para validar definitivamente.

A ideia é testar valores de 0.1 a 1 para ver como isso influencia no resultado.

```
alpha_list <- seq(0.05, 1, 0.05)

acuracia_list <- c()

for(alpha in alpha_list){
  cross_validation_set <- cross_validation_set %>%
    mutate(predicao = map_chr(sms, function(mensagem) {classificar_mensagem(mensagem,
    matriz <- cross_validation_set %>% select(label, predicao) %>% table()
```

```

# matriz %>% print()
acuracia <- (matriz[1,1] + matriz[2,2]) / sum(matriz)

acuracia_list <- c(acuracia_list, acuracia)

}

resultado <- tibble(alpha = alpha_list,
                    acuracia = acuracia_list)
resultado

```

```

## # A tibble: 20 x 2
##   alpha acuracia
##   <dbl>   <dbl>
## 1  0.05    0.15
## 2  0.1     0.15
## 3  0.15    0.15
## 4  0.2     0.15
## 5  0.25    0.15
## 6  0.3     0.15
## 7  0.35    0.15
## 8  0.4     0.15
## 9  0.45    0.15
## 10 0.5     0.15
## 11 0.55    0.15
## 12 0.6     0.15
## 13 0.65    0.15
## 14 0.7     0.15
## 15 0.75    0.15
## 16 0.8     0.15
## 17 0.85    0.15
## 18 0.9     0.15
## 19 0.95    0.15
## 20 1       0.15

```

Talvez pelo fato de esta base ser muito pequena, foi difícil evidenciar a influência do alpha no resultado final.

Na base teste chegamos na mesma acurácia percebida durante o desenvolvimento, o que demonstra estabilidade no resultado.

```
test_set <- test_set %>%  
mutate(predicao = map_chr(sms, function(mensagem) {classificar_mensagem(mensagem, 1)})  
  
matriz <- test_set %>% select(label, predicao) %>% table()  
matriz %>% print()
```

```
##      predicao  
## label  ham spam  
##   ham    4   85  
##   spam    0   11
```

```
acuracia <- (matriz[1,1] + matriz[2,2]) / sum(matriz)  
acuracia
```

```
## [1] 0.15
```